Chapter 18

Finding Vulnerabilities in Source Code:

Source code auditing is an alternative approach to finding vulnerabilities, involving reviewing application code instead of interacting with a live system. It can be done when code is accessible through open-source repositories, penetration testing permissions, file disclosure flaws, or client-side scripts like JavaScript. Even with limited programming knowledge, testers can identify common vulnerabilities using a standard methodology and language-specific cheat sheets.

Approaches to Code Review:

Aspect	**Black-Box Testing**	**White-Box Testing**
Definition	Tests the application from the outside by analyzing inputs and outputs without knowledge of internal workings	Tests the application internally with full access to source code, design, and documentation
Approach	External attack-based method	Internal code review method
Knowledge Required	No prior knowledge of code or structure	Full knowledge of code and system internals
Effectiveness	Quickly finds many practical vulnerabilities using techniques like fuzzing	Identifies deep or hidden issues that are hard to detect externally
Speed	Fast—can test hundreds of cases per minute	Slower—requires manual code analysis
Example	Sending crafted inputs to detect vulnerabilities	Finding backdoor passwords directly in source code
Limitations	May miss hidden or logic-based vulnerabilities	Time-consuming and may miss issues better found via testing
Best Use	Discovering common and surface-level vulnerabilities efficiently	Finding complex, hidden, or logic flaws in code
Relationship	Complements white-box testing	Complements black-box testing
Conclusion	Best when combined with white-box for effective security assessment	Best when combined with black-box for thorough analysis

Code Review Methodology:

Code review methodology aims to find maximum vulnerabilities in limited time using a structured approach. It involves tracing user-controlled data, searching for common vulnerability patterns, and closely reviewing high-risk code areas like authentication and input handling. Understanding language behavior and considering custom implementations helps make the review more effective.

Signatures of Common Vulnerabilities:

➔ **Cross-Site Scripting:**

Cross-site scripting (XSS) occurs when user-controlled data is directly included in HTML responses without proper encoding, allowing attackers to inject malicious scripts. Simple encoding fixes may fail if the data is already embedded within valid HTML, and filter-based defenses are often unreliable. In some cases, user input is stored in variables and later used in responses, requiring careful tracing to ensure proper sanitization. Code review is especially useful for detecting such hidden XSS flaws that may not be easily found through testing.

Code Snippet:

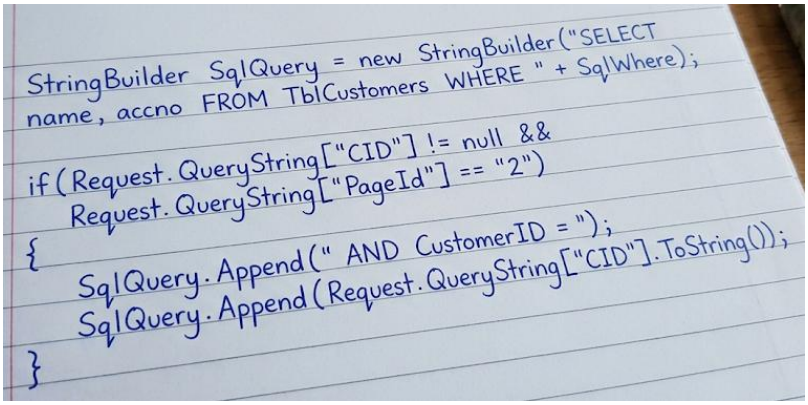
Java

```
String link = "<a href=" + HttpUtility.UrlDecode(Request.QueryString["refURL"])
+ "&SiteID=" + SiteId + "&Path="
+ HttpUtility.UrlEncode(Request.QueryString["Path"]) + "</a>";
objCell.InnerHtml = link;

private void setPageTitle(HttpServletRequest request) throws ServletException {
    String requestType = request.getParameter("type");
    if ("3".equals(requestType) && null != request.getParameter("title"))
        m_pageTitle = request.getParameter("title");
    else
        m_pageTitle = "Online banking application";
}
```

➔ **SQL Injection**

SQL injection occurs when SQL queries are built by concatenating hard-coded strings with user-controlled input, allowing attackers to manipulate database queries. These vulnerabilities are often easy to spot by searching for SQL keywords (like SELECT, INSERT, WHERE) combined with user input in the code. Reviewers must check whether such concatenations exist and if proper safeguards are missing. Code review helps quickly identify these “low-hanging” SQL injection flaws.



➔ **Path Traversal**

Path traversal vulnerabilities occur when user-controlled input is directly used in file system operations without proper validation, allowing attackers to access unintended files. Typically, input is appended to a directory path, enabling the use of sequences like ../ to navigate outside the intended directory. Code review should focus on file upload/download features and how user input is passed to file APIs. Searching for filename-related parameters and file-handling functions helps quickly identify such issues.

Code Snippet:

Java

```java
public byte[] GetAttachment(HttpRequest Request)
{
    FileStream fsAttachment = new FileStream(SpreadsheetPath +
    HttpUtility.UrlDecode(Request.QueryString["AttachName"]),
    FileMode.Open, FileAccess.Read, FileShare.Read);

    byte[] bAttachment = new byte[fsAttachment.Length];
    fsAttachment.Read(FileContent, 0,
    Convert.ToInt32(fsAttachment.Length, CultureInfo.CurrentCulture));

    fsAttachment.Close();
    return bAttachment;
}
```

➔ **Arbitrary Redirection**

Arbitrary redirection is a phishing technique where attackers manipulate a website to redirect users to malicious links. This usually happens when user input from a URL is directly used to create a redirect link without proper validation. Even if developers try to block unsafe links, improper handling like decoding data after validation can still allow attackers to bypass checks. As a result, users can be unknowingly redirected to harmful external websites.

```javascript
url = document.URL;
index = url.indexOf('?redir=');
target = unescape(url.substring(index + 7, url.length));
target = unescape(target);

if ((index = target.indexOf('//')) > 0) {
  target = target.substring(index + 2, target.length);
  index = target.indexOf('/');
  target = target.substring(index, target.length);
}

target = unescape(target);
document.location = target;
```

➔ **OS Command injection**

OS Command Injection occurs when a program uses user input directly in system commands without proper validation. In this example, the message and addr values come from user input and are inserted into a command executed by system(). This allows attackers to modify the command and run harmful operations on the system. As a result, they can execute unauthorized commands on the server.

```c
void send_mail(const char *message, const char *addr)
{
  char sendMailCmd[4096];
  snprintf(sendMailCmd, 4096, "echo '%s' | sendmail %s", message, addr);
  system(sendMailCmd);
  return;
}
```

➔ **Backdoor Passwords**

Backdoor passwords are hidden or hardcoded passwords left in the code, often for testing or admin use, which can bypass normal authentication. In this example, even if the entered password is incorrect, access is still granted if it matches the hardcoded password "oculiomnium". Such flaws are easy to spot in code and can allow unauthorized users to log in. Other similar issues include unused functions or hidden debug parameters.

Code example:

```java
private UserProfile validateUser(String username, String password)
{
  UserProfile up = getUserProfile(username);
  if (checkCredentials(up, password) ||
      "oculiomnium".equals(password))
    return up;
  return null;
}
```

Native Software Bugs:

Any native code used by the application should be closely reviewed for classic vulnerabilities that may be exploitable to execute arbitrary code. Some are:

i. Buffer Overflow Vulnerabilities
ii. Integer Vulnerabilities
iii. Format String Vulnerabilities

➔ Buffer overflow vulnerabilities occur when a program copies data into a fixed-size buffer without checking its size, which can overwrite memory and cause crashes or attacks. This often happens when unsafe functions like strcpy are used with user-controlled input. In the example, the input pszName is copied into a buffer without validation, making it vulnerable. Even "safer" functions like strncpy can still be misused if proper size checks are not done.

➔ Integer vulnerabilities happen when programs incorrectly handle numbers, especially when mixing signed and unsigned integers. In this example, a signed variable len is compared with an unsigned value sizeof(strFileName). If len is negative, the condition may still pass, allowing unsafe operations like strcpy to execute. This can lead to security issues such as buffer overflows.

➔ Format string vulnerabilities occur when user input is used directly as a format string in functions like printf or fprintf. In this example, the formatted string tmp (which includes user input username) is passed directly to fprintf without specifying a fixed format. This allows attackers to manipulate the format string and potentially read or modify memory. Such issues can lead to serious security risks.

The Java Platform:

Identifying User-Supplied Data:

The Java platform provides several ways for web applications to handle user input and interact with sessions, mainly through the *javax.servlet.http.HttpServletRequest* and *javax.servlet.ServletRequest* interfaces. These interfaces include APIs that retrieve user-supplied data from query strings, POST bodies, headers, cookies, and raw request streams. Methods such as *getParameter, getHeader*, and *getCookies* allow applications to access different parts of an HTTP request. Since this data originates from users, it can be potentially malicious, making careful handling essential for maintaining application security.

Session Interaction:

Java applications manage user session data using the javax.servlet.http.HttpSession interface, which allows information to be stored and retrieved for each user session. Session storage works as a mapping of string names to object values, enabling flexible data handling. APIs like setAttribute and putValue are used to store data, while getAttribute, getValue, and related methods retrieve stored session information. This mechanism helps maintain user-specific state across multiple requests in a web application.

Potentially Dangerous APIs:

File Access:

Some Java APIs, particularly those related to file handling, can introduce security risks if used improperly. The java.io.File class is commonly used to access files and directories, but passing unchecked user input as a filename can lead to path traversal vulnerabilities. Attackers may exploit this by using sequences like "..\" to access sensitive files outside the intended directory. Similarly, classes such as java.io.FileInputStream, java.io.FileOutputStream, java.io.FileReader, and java.io.FileWriter can also be unsafe if they use user-controlled file paths without proper validation, making secure input handling essential.

Database Access:

Certain Java database APIs, such as java.sql.Statement.executeQuery and java.sql.Connection.createStatement, can lead to SQL injection vulnerabilities when user input is directly included in query strings without validation. Attackers can manipulate queries to bypass authentication or access unauthorized data. A safer approach is to use prepared statements via java.sql.Connection.prepareStatement and related methods like java.sql.PreparedStatement.setString, which separate SQL logic from user input. This ensures inputs are handled securely and prevents malicious query manipulation.

Dynamic Code Execution:

Java does not natively support dynamic execution of source code, though some implementations may provide such functionality. If an application generates and executes Java code at runtime, it is important to examine how this is implemented. Special care must be taken to ensure that user-controlled input is not incorporated unsafely into the generated code, as this could lead to serious security vulnerabilities.

OS Command Execution:

Java applications can execute operating system commands using APIs like java.lang.Runtime.exec and java.lang.Runtime.getRuntime, but improper use can lead to serious security risks. If user input fully controls the command string, it may allow arbitrary command execution, enabling attackers to run malicious programs. Even when users control only part of the input, vulnerabilities may still arise depending on how the command is constructed. Therefore, strict validation and safe handling of user input are essential to prevent command injection and related attacks.

URL Redirection:

Java applications can perform HTTP redirection using APIs like javax.servlet.http.HttpServletResponse.sendRedirect, as well as javax.servlet.http.HttpServletResponse.setStatus and javax.servlet.http.HttpServletResponse.addHeader. If the redirect URL is controlled by user input, it can create a security risk by enabling phishing attacks through malicious redirects. Since redirects can also be implemented by setting a 3xx status code and a Location header, these alternative methods should be carefully reviewed. Proper validation of redirect URLs is necessary to prevent abuse.

Sockets:

The java.net.Socket class is used to create network connections by specifying a target host and port. If these parameters are influenced by user input, the application may become vulnerable to misuse, allowing attackers to initiate connections to arbitrary external or internal systems. This can expose internal networks or services, making proper validation and restriction of user-supplied connection details essential for security.

--The End--